Fundit V2: crowdfunding escrow on Arc

Version 2.0 draft, October 2026. Prepared for the Circle Developer Grants program.

Abstract

Fundit is a crowdfunding app where each fundraiser is its own escrow contract on Arc, Circle's stablecoin-native blockchain. Donations arrive in USDC or EURC. The creator withdraws to Arc, or receives USDC on another chain through CCTP V2. The platform operator cannot move or freeze any campaign's funds. Fundit runs as a static web app with no backend, no database and no server that holds spending keys. This paper explains the design, the trust model and the plan for launch.

1. Problem

Online fundraising platforms hold donor money in their own accounts before they pay it out. That model has three costs for the people who use it:

- **Custody risk.** The platform can freeze, delay or lose funds. Donors cannot check where their money is.
- **Slow and local payouts.** Creators wait days for a bank transfer, and cross-border payouts add fees and delays.
- **Opaque books.** Donors see a total on a web page. They cannot audit what came in and what went out.

Earlier on-chain fundraising tools fixed some of this but added new problems. Many pooled funds in an operator vault, relied on bridges with their own trust assumptions, or needed a backend that held private keys. The first version of Fundit had these flaws. It sent donations to an owner vault, kept deposit-wallet keys in a database and paid creators off chain. Fundit V2 is a full rebuild that removes all of it.

2. Design goals

1. **Zero custody.** Funds sit in a per-campaign contract. No operator key can move them.
2. **Stablecoins only.** USDC and EURC, so a donation keeps its value between giving and payout.
3. **One home chain.** Every campaign lives on Arc. Donors and creators on other chains connect through Circle's cross-chain products, never through a third-party bridge.
4. **No servers with keys, no paid infrastructure.** The app is static. All reads come from public RPC endpoints and on-chain events.
5. **Honest claims.** A feature ships to users only after a live mainnet check confirms it.

3. Why Arc and Circle

Arc is a Layer 1 blockchain from Circle where USDC is the gas token. Donors and creators hold one asset and never need a separate token for fees. Arc mainnet (chain ID `5042`) is a CCTP V2 domain (`26`), and Circle's Gateway, Arc Memo and EURC are live on it.

Fundit uses these Circle and Arc products:

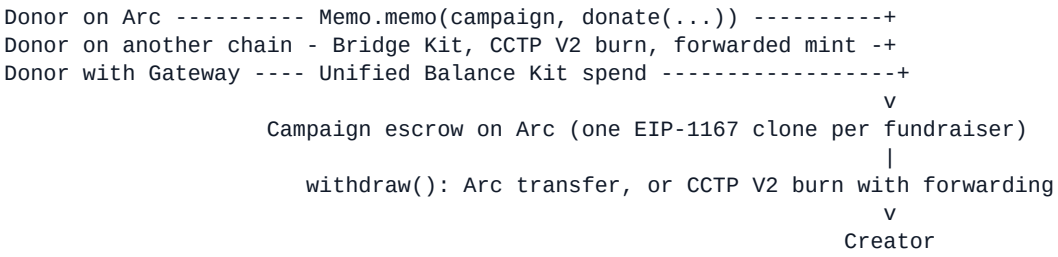
Product	Role in Fundit
Arc	Home chain for every campaign escrow.
USDC and EURC	The two campaign tokens, used through the ERC-20 interface with 6 decimals.
Arc Memo	Tags each on-Arc donation from an EOA, while keeping the donor as `msg.sender`.
CCTP V2 and the Forwarding Service	Creator payouts in USDC to six other chains, in one transaction.
Bridge Kit	Donations from other chains, minted straight into the campaign.
Gateway and Unified Balance Kit	Donations from a donor's unified USDC balance.
Modular Wallets	Passkey smart accounts, so new users need no browser extension.

Bridge Kit, Gateway and Modular Wallets are built and tested on an Arc mainnet fork. They are behind build flags at launch and turn on after the live canary in section 9 confirms them.

4. System overview

Fundit has two parts:

- **Contracts.** `CampaignFactory` and `Campaign`, written in Solidity and tested with Foundry.
- **App.** A React and TypeScript single-page app, served as static files from Cloudflare Pages.



4.1 Factory

`CampaignFactory` deploys each campaign as an EIP-1167 minimal clone with CREATE2. The salt is the creator's address and a per-creator nonce, so the app can predict a campaign address before it exists. The factory fixes the USDC, EURC and `TokenMessengerV2` addresses in its constructor. The owner cannot add tokens, which keeps fee-on-transfer and rebasing tokens out of the accounting.

At creation the factory copies the current donation fee and withdrawal fee into the new campaign. Later fee changes apply only to campaigns created after the change.

4.2 Campaign escrow

Each `Campaign` holds one token and tracks four counters:

- `grossDonated`: donation amounts before fees, tips excluded.
- `creatorBalance`: the net amount the creator can withdraw.
- `feesAccrued`: donation fees, withdrawal fees and tips not yet claimed.

- `withdrawn`: the total the creator has taken out.

The contract keeps this invariant at all times:

```
token.balanceOf(campaign) == creatorBalance + feesAccrued + unaccounted
```

`unaccounted` is any campaign token that arrived without a `donate` call: a CCTP mint, a Gateway mint or a plain transfer. `settle()` books it as a gross donation and charges the donation fee. `donate`, `withdraw` and `claimFees` settle first, so every inflow pays the same fee no matter how it arrived. The invariant is checked by Foundry invariant tests.

`raised()` returns `grossDonated` plus any unaccounted amount. Withdrawals never reduce it, so the progress bar always shows the total raised.

4.3 Donations on Arc

The donor approves the campaign for `amount + tip`, then calls `donate(amount, tip, message)`. When the donor uses a regular wallet (an EOA), the app wraps the call in `Memo.memo(target, data, memoId, memoData)` with the memo ID `keccak256("fundit.donate.v1")`. A fork test and a live `eth_call` confirmed that Memo preserves `msg.sender`, so the campaign records the donor's own address. Smart accounts and EIP-7702 delegated addresses call the campaign directly, because Memo accepts only EOA callers.

4.4 Donations from other chains

A donor on Ethereum, Avalanche, OP Mainnet, Arbitrum, Base or Polygon uses Bridge Kit to burn USDC through CCTP V2. The mint recipient is the campaign contract on Arc, and the CCTP Forwarding Service submits the mint, so the donor needs no Arc gas. The minted USDC is unaccounted until the next `settle()`.

A donor with a Gateway unified USDC balance spends from it with the Unified Balance Kit. The recipient is again the campaign address on Arc.

Whether a contract works as a forwarded `mintRecipient` is the main open question for these paths. The Forwarding Service docs describe the recipient as a wallet address. The fork tests and the end-to-end tests cover the app side with a stubbed attestation. The live canary settles the question before the flag turns on.

4.5 Withdrawals and payouts

Only the creator can call `withdraw(amount, destDomain, recipient, maxFee, minFinalityThreshold, hookData)`. The contract settles, takes the withdrawal fee, then:

- On Arc (domain 26), transfers the net amount to the recipient.
- On another CCTP domain, approves exactly the net amount to `TokenMessengerV2` and calls `depositForBurnWithHook` with the Forwarding Service hook data. Circle mints on the destination chain, and the creator pays no gas there.

The app quotes the Fast and Standard fees from Circle's CCTP fees API before it builds the call. A fork test confirmed a USDC withdrawal to Base (domain 6) against the real `TokenMessengerV2` on Arc mainnet.

4.6 EURC campaigns

EURC campaigns take donations and pay out on Arc only. On Arc mainnet, CCTP cannot burn EURC today. On a fork, `depositForBurn(EURC)` to domains 0, 1, 6 and 14 reverts with "Burn token not

supported", and `TokenMinterV2.burnLimitsPerMessage(EURC)` is 0. Press reports said EURC CCTP was live on Arc. Fundit follows on-chain state instead.

The factory takes a list of EURC payout domains in its constructor, with no setter. The launch list is empty. A creator can still withdraw EURC on Arc and move it themselves. Adding an EURC route later needs a new factory.

4.7 The app

The app reads everything from chain. The home page and explore page page through the factory and multicall each campaign's `summary()`. Donation history comes from `eth_getLogs` on the campaign address. Every campaign event ends with `prevActivityBlock`, a pointer to the previous block with campaign events. The app walks backwards in 10,000-block windows, the limit of the public Arc RPC, and skips quiet blocks.

Each transaction waits for its receipt before the app shows success. The app estimates gas on Arc and adds a 25% margin. Names come from ENS and Basenames. Campaign images are links, with no uploads.

Hosting is Cloudflare Pages. A content security policy limits scripts and network calls to the Arc RPC, the CCTP chain RPCs, Reown and Circle's APIs.

5. Trust model

5.1 What the owner can do

The factory has one owner, set at deploy, with `Ownable2Step` transfers. The owner can:

- Set the donation fee and withdrawal fee for new campaigns, each at most 5% (`MAX_FEE_BPS = 500`).
- Set the treasury that receives claimed fees.
- Set a featured campaign.
- Pause and unpause the creation of new campaigns.

`renounceOwnership` always reverts, so the treasury can always be rotated.

5.2 What the owner can never do

- Move, freeze or redirect campaign funds.
- Pause donations, withdrawals, settlement or fee claims on an existing campaign.
- Change an existing campaign's fees, token, creator or goal.
- Add a token, change the messenger or upgrade any contract.
- Take part in creator recovery.

5.3 Fees

The launch configuration is a 1% donation fee and a 1% withdrawal fee. Each fee rounds down in the creator's favor. Tips are optional and chosen by the donor. Fees stay in the campaign until anyone calls `claimFees()`. A treasury that reverts can block only fee claims, never donations or withdrawals.

5.4 Creator recovery

A creator who loses their key would lock the campaign balance forever. To prevent that, a creator can name an optional backup address. The backup calls `initiateRecovery`, waits 7 days (`RECOVERY_DELAY`) and then calls `completeRecovery` to become the creator. During the wait the creator keeps every power and can cancel with `cancelRecovery`. Changing the backup also cancels a pending recovery. No recovery function moves tokens.

5.5 Wrong-token rescue

If someone sends a token other than the campaign token, the creator can call `sweep(token, to)` to recover it with no fee. The call reverts if the campaign-token balance drops, which guards against a second address that controls the same asset.

6. Security process

- **Tests.** Foundry unit, fuzz and invariant tests. The invariants cover the balance equation, fee caps, owner powers and a reverting treasury.
- **Fork tests.** An Arc mainnet fork suite runs under Arc Foundry `v0.8.0-2`. It covers the CCTP payout path, Memo sender preservation, EIP-3009 and EIP-2612 entry points, and EURC CCTP support. A strict mode turns those findings into assertions right before the deploy.
- **Static analysis.** Slither, with every accepted finding listed in `contracts/SECURITY.md`.
- **End-to-end tests.** Playwright drives the built app against a local Arc mainnet fork for create, donate, withdraw and a stubbed cross-chain donation.
- **Canary.** A small live run on mainnet before any flag turns on.

The contracts have not had a third-party audit. A grant would fund one (section 10).

7. Business model

Fundit earns the donation fee, the withdrawal fee and optional tips. All three are visible on chain and in the donate form before the donor signs. The fees for a campaign never change after it is created.

Fundit has no infrastructure bill. The app is static, hosting is on a free tier, and all reads use public RPC endpoints.

8. Status

As of October 2026:

- Contracts: built and tested. Not deployed to Arc mainnet.
- App: built and tested against an Arc mainnet fork. Not deployed.
- Cross-chain donations, unified balance donations and passkeys: built, behind flags.
- Traction: none yet. Fundit has not launched. Metrics in this paper are placeholders until launch.

9. Launch plan

1. Run the strict fork gate and print the exact deploy configuration for review.
2. Deploy the factory to Arc mainnet and verify it on the Arc explorer.
3. Deploy the app to `fundit.xyz` on Cloudflare Pages with both flags off.

4. Run a canary with about 5 USDC: a donation and a withdrawal on Arc, a donation from Base through Bridge Kit, and a payout to Base through CCTP. A unified balance donation and a passkey donation are optional steps.
5. Turn on `VITE_CROSSCHAIN` and `VITE_PASSKEYS` only for the paths the canary confirms.

10. Roadmap and grant milestones

The milestones below are proposed for a Circle Developer Grant paid in USDC on delivery. Targets in brackets are placeholders to agree with the program.

Milestone	Deliverable	Evidence
M1: Mainnet launch	Factory deployed and verified, app live, canary complete	Arc explorer links, canary transaction list
M2: Cross-chain and passkeys on	`VITE_CROSSCHAIN` and `VITE_PASSKEYS` enabled after the canary	Live transactions for each path
M3: Audit	Third-party review of `Campaign` and `CampaignFactory`, fixes in a new factory if needed	Published report
M4: Adoption	[placeholder: number of campaigns] campaigns and [placeholder: USDC volume] raised	On-chain factory data
M5: V2 card donations	Onramp Kit card donations into campaigns	Live campaign with a card donation

Agent features (for example, an agent that drafts a campaign page from a short brief) are a later idea. No agent will hold a key that controls money.

11. Risks

- **Forwarded mint into a contract.** If the Forwarding Service does not mint to a contract recipient, cross-chain donations stay off. Payouts are not affected, because they mint to the creator's wallet.
- **Fixed configuration.** A wrong fee or EURC payout list cannot be corrected for existing campaigns after the deploy. The deploy gate prints the exact configuration for review before the deploy.
- **Smart account payouts.** A passkey account may not exist on other chains. The app defaults passkey creators to Arc payouts and warns before another chain.
- **No audit yet.** Mitigated by tests, fork tests, Slither and a small canary. An audit is milestone M3.

References

- Contracts: `contracts/src/Campaign.sol`, `contracts/src/CampaignFactory.sol`
- Trust model: `contracts/SECURITY.md`
- Product list in the app: `app/src/pages/Home.tsx`
- Circle developer docs: https://developers.circle.com